

Drone Autonome Stabilise sur Microcontrôleur STM32F446RE

Conception et implementation d'un quadricoptere autonome avec
architecture Real Time Operating System, filtre Madgwick et correcteur PID
multi-axes

Terki Boubkeur

Systemes Embarques

Mai 2026

STM32F446RE	Cortex-M4 180MHz	Real Time OS	FPU Hardware
MPU6050 I2C	Filtre Madgwick	PID 3 axes	PWM ESC 50Hz
Langage C	UART 115200bps	MATLAB Simulink	Chassis F450

1. Introduction

Ce projet a pour objectif la conception complete d'un drone quadricoptere autonome, depuis la lecture des capteurs inertiels jusqu'au controle des moteurs brushless en temps reel. L'ensemble du systeme est developpe en langage C embarque sur microcontrolleur STM32F446RE (Cortex-M4, 180 MHz).

Le defi principal reside dans la contrainte temps reel : la lecture du capteur MPU6050, le calcul du filtre Madgwick, les corrections PID et la mise a jour simultanee des 4 moteurs brushless doivent s'executer de maniere concurrente et synchronisee, avec une latence totale inferieure a 10 ms.

Objectif : stabiliser automatiquement un drone en vol en fusionnant les donnees inertielles d'un MPU6050 et en calculant des corrections PID en temps reel.

1.1 Dynamique du drone

Un drone quadricoptere est soumis a plusieurs forces et moments. La stabilisation consiste a controler independamment les 3 axes de rotation Roll, Pitch et Yaw ainsi que l'altitude Z.

Key Forces and Components of Forces in Drone Flight:

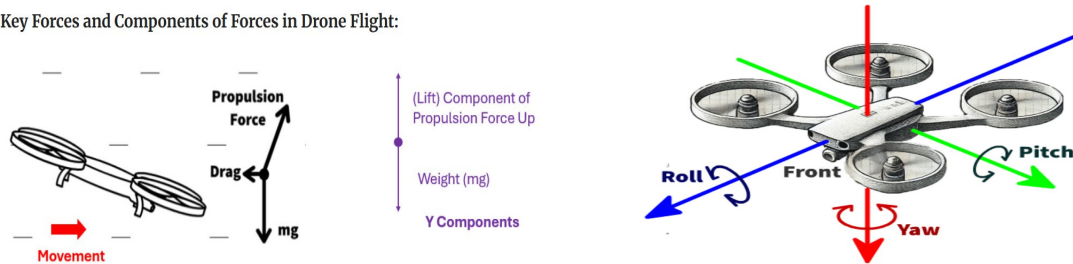


Figure 1 — Forces appliquees au drone en vol (gauche) et axes de rotation Roll/Pitch/Yaw (droite)

2. Architecture Materielle

2.1 Composants principaux

Composant	Specification	Interface
STM32 Nucleo F446RE	Cortex-M4 @ 180 MHz - FPU - 512KB Flash	USB / ST-Link
MPU6050 GY-521	Gyro 250 deg/s - Accelero 2g	I2C @ 100 kHz
ESC 30A x 4	Signal PWM 50 Hz - 1000-2000 us	PWM TIM2
Moteurs 2212 920KV	Brushless - Helices 1045 CW/CCW	Cable 3 fils
LiPo 3S 2200 mAh	11.1 V nominal - connecteur XT60	Puissance directe
UBEC 5V 3A	Regulateur abaisseur	5V vers STM32

2.2 Interfaces de communication

```
MPU6050 --> STM32 I2C (PB8=SCL, PB7=SDA, 100 kHz)

PC/PuTTY --> STM32 UART (PA2=TX, PA3=RX, 115200 bps)

STM32 --> ESC x 4 PWM (PA0..PA3, TIM2 CH1-4, 50 Hz)

LiPo --> ESC x 4 DC (11.1 V, courant max 30 A/ESC)
```

3. Real Time Operating System

Le systeme repose sur un Real Time Operating System qui orchestre l'execution concurrente de plusieurs taches independantes avec des priorites definies. Cette architecture garantit le respect strict des contraintes temporelles critiques pour la stabilisation du drone.

3.1 Architecture des taches

Tache	Priorite	Frequence	Role
TaskIMU	Haute	100 Hz	Lecture MPU6050 + filtre Madgwick
TaskPID	Haute	100 Hz	Calcul corrections Roll/Pitch/Yaw
TaskMoteurs	Haute	100 Hz	Mise a jour PWM 4 ESC
TaskUART	Normale	Evenement	Reception commandes clavier
TaskAffichage	Basse	5 Hz	Monitoring UART debug

3.2 Pipeline de donnees

```
TaskIMU --[QueueIMU]--> TaskPID --[QueuePID]--> TaskMoteurs

|

TaskUART --[Mutex]--> g_thro / g_pitch / g_yaw |

| |

+-----> TaskPID -----+
```

4. Filtre Madgwick - Fusion de Donnees Inertielles

4.1 Problematique

Un gyroscope mesure la vitesse angulaire (deg/s), non l'angle absolu. L'integration directe du signal gyroscopique accumule une derive temporelle rendant la stabilisation impossible a long terme. L'accelerometre seul est trop sensible aux vibrations mecaniques.

4.2 Algorithme de fusion

Le filtre Madgwick fusionne les donnees du gyroscope et de l'accelerometre via une representation en quaternions, offrant une estimation d'orientation precise sans singularite de calcul.

Equation de mise a jour :

$$\mathbf{q_dot} = (1/2) * \mathbf{q} * \mathbf{\omega_gyro} - \mathbf{\beta} * \mathbf{gradient_f}(\mathbf{q}, \mathbf{a})$$

Conversion quaternion --> angles Euler :

$$\text{Roll} = \text{atan2}(2(q_0q_1 + q_2q_3), 1 - 2(q_1^2 + q_2^2)) * 57.296$$

$$\text{Pitch} = \text{asin}(2(q_0q_2 - q_3q_1)) * 57.296$$

$$\text{Yaw} = \text{atan2}(2(q_0q_3 + q_1q_2), 1 - 2(q_2^2 + q_3^2)) * 57.296$$

4.3 Resultats experimentaux

```
<== demarrage ==>
<== test i2c ==>
trouve 0x68
peripherique I2C OK
PWM démarre
Armement ESC attend
<== demarrage ==>
<== test i2c ==>
trouve 0x68
peripherique I2C OK
PWM démarre
Armement ESC attend
```

Figure 2 - Terminal PuTTY : démarrage, scan I2C, detection MPU6050 (0x68) et armement ESC

Mesure	Valeur	Statut
Offset gyroscope X (avant calibration)	-4.27 deg/s	Corrige
Offset gyroscope Y (avant calibration)	-0.15 deg/s	Corrige
Offset gyroscope Z (avant calibration)	+1.00 deg/s	Corrige
Derive residuelle apres Madgwick	< 0.1 deg/s	Excellent
Precision angulaire au repos	+/- 0.5 deg	Stable
Nombre de mesures calibration	200 mesures	Precis

5. Algorithme PID - Correcteur Multi-Axes

5.1 Principe du correcteur PID

Trois correcteurs PID independants assurent la stabilisation sur chacun des axes de rotation du drone (Roll, Pitch, Yaw). Chaque correcteur recoit l'angle mesure par le filtre Madgwick et calcule la correction de poussee differentielle necessaire.

```
correction(t) = Kp * e(t) + Ki * integral(e) + Kd * de/dt
```

```
Avec : e(t) = consigne - mesure(t)
```

```
Anti-windup integral : |integral| <= 10.0
```

```
Saturation sortie : |correction| <= 100.0
```

```
Parametres calcules par methode du 2eme ordre (zeta=0.7, tr=2s) :
```

```
pid_roll --> Kp=260 Ki=0 Kd=180
```

```
pid_pitch --> Kp=260 Ki=0 Kd=180
```

```
pid_yaw --> Kp=100 Ki=0 Kd=50
```

5.2 Mix moteurs - Quadricoptere en croix (+)

Moteur	Position	Rotation	Throttle	Roll	Pitch	Yaw
M1	Avant-Gauche	CCW	+	+	+	-
M2	Avant-Droite	CW	+	-	+	+
M3	Arriere-Gauche	CW	+	+	-	+
M4	Arriere-Droite	CCW	+	-	-	-

5.3 Resultats experimentaux

```
-- Drone a plat, immobile -----
```

```
THR: 0.0 ROL: 0.01 PIT: 0.00 YAW: 0.00
```

M1:500 M2:500 M3:500 M4:500 <- arret ESC

-- Throttle 44% + penche gauche (ROL = +13.5 deg) -----

THR:44.0 ROL:13.50 PIT: 0.02 YAW:-0.01

M1:672 M2:560 M3:664 M4:582 <- M1/M3 > M2/M4 OK

-- Throttle 44% + penche avant (PIT = +9.3 deg) -----

THR:44.0 ROL: 0.11 PIT: 9.33 YAW: 0.00

M1:628 M2:631 M3:589 M4:585 <- M1/M2 > M3/M4 OK

6. Simulation MATLAB / Simulink

6.1 Objectif de la simulation

Un modele dynamique complet du drone a ete developpe sous MATLAB/Simulink pour valider les gains PID et etudier la stabilite du systeme en boucle fermee avant deploiement sur le materiel reel.

6.2 Equations dynamiques

Rotation :

$$I_{xx} \cdot \ddot{\phi} = L \cdot (F_2 + F_4 - F_1 - F_3) \leftarrow \text{Roll}$$

$$I_{yy} \cdot \ddot{\theta} = L \cdot (F_1 + F_2 - F_3 - F_4) \leftarrow \text{Pitch}$$

$$I_{zz} \cdot \ddot{\psi} = km \cdot (F_1 - F_2 + F_3 - F_4) \leftarrow \text{Yaw}$$

$$\text{Modele moteur : } F_i = k_t \cdot \omega_i^2$$

Parametres : $m=1.0\text{kg}$ $L=0.225\text{m}$ $I_{xx}=I_{yy}=0.0145 \text{ kg.m}^2$

$k_t=1.269\text{e-}5$ $km=1.269\text{e-}7$

6.3 Architecture du modele Simulink

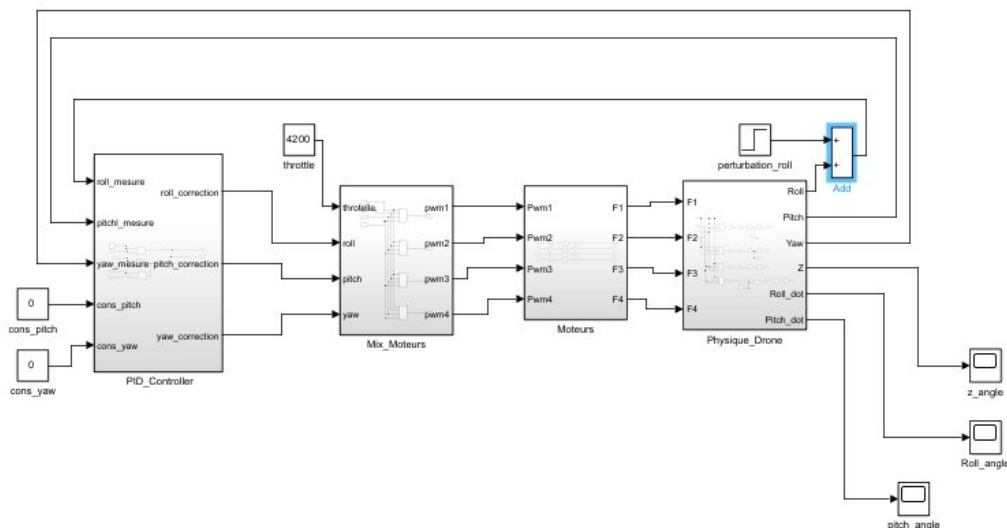


Figure 3 - Modele Simulink complet en boucle fermee : PID_Controller -> Mix_Moteurs -> Moteurs -> Physique_Drone

6.4 Calcul des gains PID optimaux

Les gains PID ont été calculés par la méthode du système du second ordre avec un amortissement $\zeta=0.7$ et un temps de réponse cible de 2 secondes :

Fréquence naturelle : $\omega_n = 4 / t_r = 4 / 2 = 2.0 \text{ rad/s}$

Amortissement : $\zeta = 0.7$ (critiquement amorti)

Gain boucle linearise : $K = (4 * L * k_t * k_{pwm}^2) / I_{xx} = 4.32e-6$

Gains calculés (modèle Simulink) :

$k_p = \omega_n^2 / K = 463266$

$k_d = 2 * \zeta * \omega_n / K = 324286$

Gains convertis pour STM32 (facteur échelle PWM) :

$k_{p_stm32} = 260$

$k_{d_stm32} = 180$

6.5 Réponse en boucle fermée

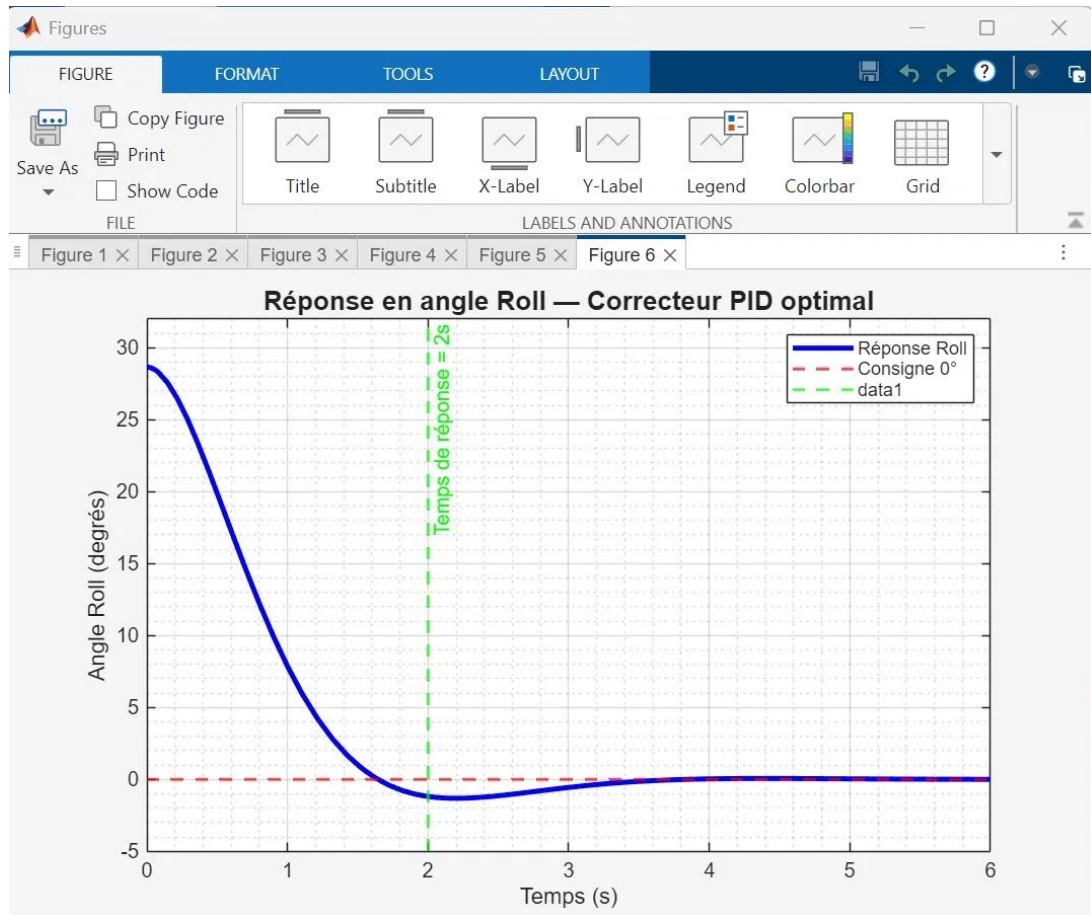


Figure 4 - Réponse en angle Roll avec correcteur PID optimal ($k_p=260$, $k_d=180$) - Temps de réponse = 2s

Le drone démarre penché à 30 degrés et le correcteur PID ramène l'angle à 0 degré en 2 secondes avec un amortissement de 0.7, ce qui correspond à une réponse rapide sans oscillations importantes.

7. Conclusion et Perspectives

7.1 Bilan du projet

Ce projet a permis de concevoir et de valider un systeme complet de stabilisation embarque sur microcontroleur STM32, alliant traitement temps reel, fusion de donnees inertielles et controle automatique en boucle fermee.

7.2 Realisations

- Detection et initialisation MPU6050 via scan I2C automatique
- Calibration automatique des offsets gyroscope (200 mesures)
- Filtre Madgwick - angles stables +/- 0.5 deg au repos
- Correcteur PID Roll/Pitch/Yaw avec anti-windup integral
- Mix moteurs quadricoptere valide experimentalement
- Armement et controle de 4 ESC brushless via PWM 50 Hz
- Architecture Real Time Operating System multi-taches
- Modele physique complet MATLAB/Simulink (6 degres de liberte)
- Calcul analytique des gains PID optimaux ($k_p=260$, $k_d=180$)
- Securite materielle : arret moteurs sur perte signal I2C

7.3 Perspectives

- | | |
|----|---|
| 01 | Integration radiocommande FlySky FS-i6 avec decodage signal PPM |
| 02 | Ajustement experimental des gains PID en conditions de vol reel |
| 03 | Ajout barometre BMP280 pour controle d'altitude en boucle fermee |
| 04 | Integration GPS pour navigation autonome et maintien de position |
| 05 | Amelioration filtre de Kalman pour estimation d'etat plus precise |